

Rodrigo Yupanqui Rubio

Ingeniería Electrónica — Sistemas embebidos, IoT e instrumentación

Pontificia Universidad Católica del Perú (PUCP) | Lima, Perú | rodrigo.yupanqui@pucp.edu.pe | linkedin.com/in/rodrigo-yupanqui | github.com/RodrigoYupanqui

PERFIL

Estudiante del décimo y último ciclo de Ingeniería Electrónica (PUCP), 2.º puesto de su promoción y perteneciente al décimo superior, con bachillerato previsto para diciembre de 2026. Practicante de Ingeniería Electrónica en DIACSA, en proyectos de instrumentación, adquisición de datos y firmware. Seleccionado en tres programas competitivos: Programa Regional de Eficiencia Energética de Siemens, Aspire Leaders Program (Harvard Business School) y Seeds for the Future (Huawei). Director de Proyectos en IEEE Computer Society PUCP.

Este documento resume tres proyectos de sistemas embebidos con comunicación inalámbrica, adquisición multisensor y control. Cada uno ocupa una página con arquitectura, formulación técnica, resultados y contribución personal. El código y la documentación completa están en GitHub.

RESUMEN DE PROYECTOS

Proyecto	Contexto	Sistema	Resultado
CrackScan	Proyecto Electrónico 1, PUCP (2025-1)	Rover autónomo: ESP32, Raspberry Pi 4B/5, LiDAR, WSS	Paper en IEEE INTERCON 2026; final del Student Poster Competition, ICACIT 2026
CubeSat 1U	Competencia Nacional de Pequeños Satélites 2024	ESP32, 6 sensores, telemetría LoRa a 433 MHz	Finalista entre 17 equipos; prueba de caída libre desde 100 m
Arpeggiator	Diseño Electrónico 2, PUCP (2024-2)	ESP32-S3, audio I²S de 16 bits, PCB propia	Prototipo funcional con gabinete en MDF y revisión de código documentada

COMPETENCIAS TÉCNICAS

Firmware y plataformas	C/C++ (Arduino-ESP32, STM32), Python; ESP32, ESP32-S3, STM32, Raspberry Pi 4B/5
Comunicaciones	LoRa, Wi-Fi, WebSocket seguro (WSS), MQTT, UART, I²C, SPI, I²S, RS485/Modbus RTU
Sensórica y adquisición	IMU, magnetómetro, GPS, BME280, INA219, LiDAR, cámaras, encoders
Control y procesamiento	Regulación PI con <i>anti-windup</i> , filtrado digital, PWM; OpenCV, YOLO, MATLAB/Simulink, LabVIEW
Diseño de hardware	EasyEDA (esquemáticos, PCB, Gerber), LTspice, Proteus, Autodesk Inventor, AutoCAD Electrical
Software y datos	FastAPI, Next.js, Git/GitHub, Linux, SQL, Power BI, NumPy/Pandas
Automatización industrial	PLC Allen-Bradley (Ladder, Studio 5000), PLC Festo, SCADA/HMI

EXPERIENCIA Y DISTINCIONES

- **DIACSA** (feb. 2026 – presente). Practicante de Ingeniería Electrónica: sistema de instrumentación LiDAR + STM32 para secciones transversales en socavones mineros; diseño de electrónica de potencia y PCB en EasyEDA.
- **IEEE Computer Society PUCP**. Director de Proyectos.
- **Jefe de práctica, Departamento de Ciencias, PUCP**. Sesiones de probabilidad y procesos estocásticos con MATLAB, Excel e IBM SPSS Statistics.
- **Publicaciones y concursos**. Paper de CrackScan presentado en IEEE INTERCON 2026 (Trujillo); proyecto seleccionado para la final del Student Poster Competition – Capstone Projects, 12th ICACIT Symposium 2026 (Pontificia Universidad Católica de Chile).

CrackScan: rover autónomo de inspección de pavimentos

Proyecto Electrónico 1, PUCP (asesor: S. E. Romero) | 2025-1

RESUMEN

CrackScan es un rover de bajo costo, construido con componentes comerciales, que detecta grietas y baches en el pavimento, reconstruye en 3D los baches detectados y navega siguiendo una línea amarilla. El sistema se organiza en tres nodos: una Raspberry Pi 5 ejecuta YOLOv11n y activa de forma selectiva el LiDAR LDROBOT D500 para estimar el volumen del bache; una Raspberry Pi 4B segmenta la línea (HSV, ROI trapezoidal, centroide ponderado) y envía un ángulo de corrección a unos 20 Hz; un ESP32 regula la velocidad de dos motores, coordina dos servos de dirección y mantiene la telemetría por WSS.

FORMULACIÓN TÉCNICA

El ESP32 muestrea los encoders cada $T_s = 0.1$ s (1050 pulsos por vuelta) y aplica un controlador PI con saturación e integrador acotado ($K_p = 1.8$, $K_i = 3.2$):

$$\omega[k] = \frac{600 n_{enc}[k]}{1050}, \quad e[k] = \omega^* - \omega[k] \quad (1)$$

$$I[k] = \text{sat}_{[0,255]}(I[k-1] + K_i e[k] T_s) \quad (2)$$

$$u[k] = \text{sat}_{[0,255]}(K_p e[k] + I[k]) \quad (3)$$

El ángulo recibido por UART se limita a $\pm 45^\circ$ y se filtra con una media móvil exponencial, con banda muerta de 1° y servos restringidos a 60° – 120° :

$$\theta_f[k] = (1 - \alpha) \theta_f[k-1] + \alpha \theta[k], \quad \alpha = 0.25 \quad (4)$$

Si el enlace UART permanece inactivo más de 200 ms, la referencia de velocidad pasa a cero. El volumen del bache se obtiene con un plano de referencia estimado por RANSAC y el mapa de profundidad d_{ij} sobre una rejilla regular: $V = \sum_{i,j} d_{ij} \Delta A_{ij}$.

RESULTADOS

Indicador	Valor
mAP@0.5 de YOLOv11n (265 imágenes de prueba)	0.78
F1 en grietas / baches	0.85 / 0.74
Inferencia en Raspberry Pi 5	163.84 ms (6.10 FPS)
Nube LiDAR procesada (reconstrucción)	7123 puntos en 0.40 s
Volumen estimado del bache evaluado	1348.73 cm ³
Intervención manual en la pista de prueba	ninguna

El modelo se entrenó con 7244 imágenes. Los 186 falsos negativos de baches son el principal modo de fallo, pues impiden activar el LiDAR. La comparación del volumen con una medición física de referencia está pendiente.

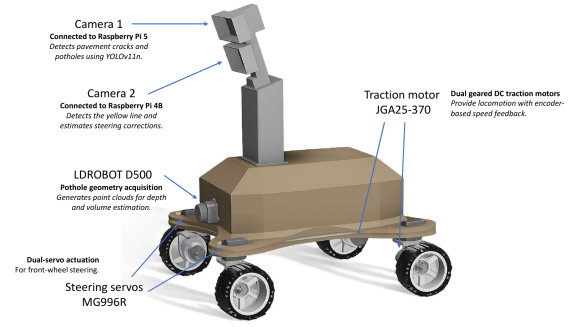


Figura 1: Plataforma: cámaras, LiDAR D500, tracción JGA25-370 y dirección con dos servos MG996R.

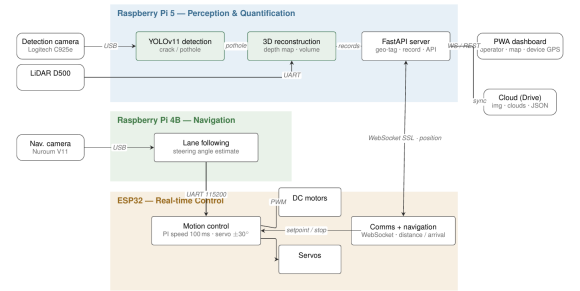


Figura 2: Arquitectura de software distribuida.

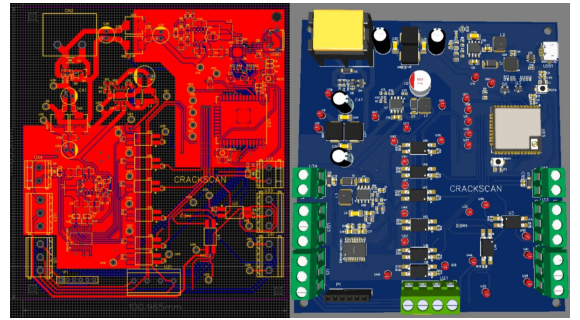


Figura 3: PCB de integración electrónica.

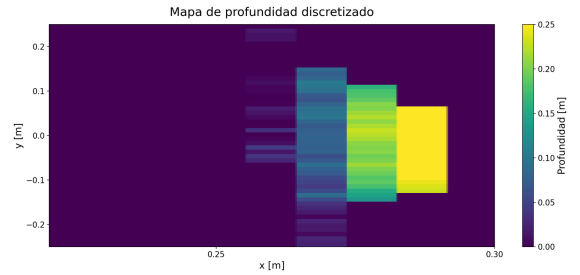


Figura 4: Mapa de profundidad LiDAR del bache evaluado (área afectada: 0.0111 m²).

CONTRIBUCIÓN PERSONAL. Coordiné la integración técnica del sistema distribuido, definiendo las interfaces entre el ESP32, las Raspberry Pi 4B/5, los sensores y los actuadores (UART y WSS con mensajes JSON). Implementé la plataforma web de monitoreo y control sobre FastAPI: control de movimiento, joystick virtual, estado del sistema, detecciones en tiempo real, registro geolocalizado con imagen y mapas interactivos. Paper presentado en IEEE INTERCON 2026; proyecto seleccionado para la final del Student Poster Competition del 12th ICACIT Symposium 2026.

CubeSat 1U: monitoreo ambiental y telemetría LoRa

Equipo Arquitectos Galácticos, Competencia Nacional de Pequeños Satélites (INRAS-PUCP, VRI-PUCP, CONIDA) | 2024

RESUMEN

Satélite de formato 1U, elevado por un dron a 100 m y liberado en caída libre. Un ESP32 adquiere temperatura, presión, humedad, aceleración, velocidad angular, campo magnético, tensión y corriente, y posición GPS (21 variables). Los datos se empaquetan en JSON y se transmiten por LoRa (SX1278, 433 MHz) a una estación terrena con interfaz gráfica en tiempo real. Durante el descenso, dos servos SG90 liberan el paracaídas. Fue uno de los 17 equipos seleccionados para la fase final presencial.

FORMULACIÓN TÉCNICA

La altitud relativa se calcula a partir de la presión del BME280 (P en hPa) con la fórmula barométrica, referida a la altitud h_0 del punto de arranque:

$$h = \frac{1 - (P/1013.25)^{1/5.2559}}{2.25577 \times 10^{-5}} - h_0 \quad [\text{m}] \quad (5)$$

El despliegue usa histéresis para evitar disparos por ruido: el sistema se arma al superar 85 m y despliega al descender por debajo de 80 m.

$$\text{armado} \leftarrow \text{armado} \vee (h > 85) \quad (6)$$

$$\text{despliegue} = \text{armado} \wedge (h < 80) \quad (7)$$

El mensaje JSON de longitud L se fragmenta en $N = \lceil L/50 \rceil$ paquetes LoRa de hasta 50 caracteres, separados por 50 ms. El receptor los reensambla y extrae cada variable. El magnetómetro se corrige por eje con $m_c = s \odot (m - b)$, donde $b = (-276, 269, -348)$ y $s = (1.08, 1.11, 0.86)$.

COMPONENTES

Módulo	Función	Bus
ESP32 DevKit V1	Dos núcleos a 240 MHz, 520 KB de SRAM	—
BME280	Presión, temperatura y humedad	I ² C
MPU6050	Acelerómetro (± 16 g) y giroscopio, 16 bits	I ² C
QMC5883L	Magnetómetro de 3 ejes (± 8 G)	I ² C
INA219	Tensión y corriente (0.8 mA, 4 mV)	I ² C
NEO-6M	GPS de 50 canales	UART
SX1278 (Ra-02)	LoRa, +20 dBm, sensibilidad -148 dBm	SPI



Figura 5: Fase final de la competencia (CNPS 2024).

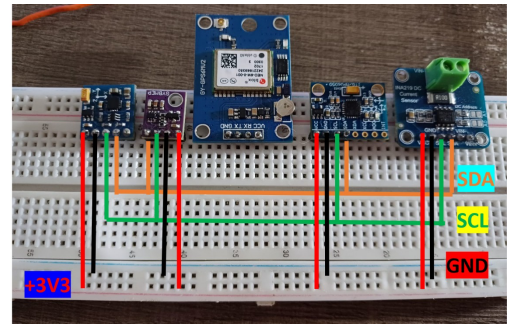


Figura 6: Validación en protoboard: bus I²C compartido y GPS.

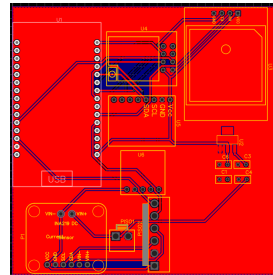


Figura 7: PCB del piso 2.

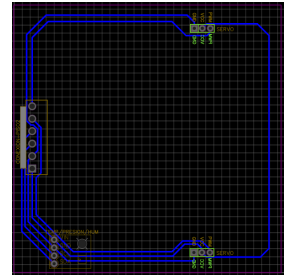


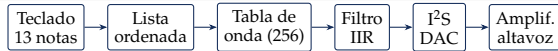
Figura 8: PCB del piso 3.

CONTRIBUCIÓN PERSONAL. Elaboré la matriz morfológica, el diagrama de flujo y los criterios de selección de componentes (precio, disponibilidad y viabilidad técnica); evalué y coticé sensores, módulos de comunicación y microcontroladores. Desarrollé el firmware en C++ para ESP32 (adquisición multisensor y transmisión LoRa) y diseñé los esquemáticos y las PCB en EasyEDA respetando las dimensiones del CubeSat. Para una siguiente iteración se recomienda alimentar con baterías Li-Ion de 3.7 V y mejorar el mecanismo de despliegue.

RESUMEN

Instrumento independiente basado en un ESP32-S3. El usuario selecciona cualquier combinación de notas en un teclado de una octava (C4–C5, 13 notas) y el equipo las reproduce como arpeggio ascendente, descendente o aleatorio, con onda senoidal, cuadrada o triangular y velocidad y tono ajustables. El audio se transmite por I²S (16 bits, 44.1 kHz) a un DAC PCM5102 y a un amplificador clase D TPA3116D2; 13 LED WS2812B indican las notas activas. El trabajo incluye PCB de doble capa y gabinete de MDF cortado con láser.

FORMULACIÓN TÉCNICA



La frecuencia de cada nota se desplaza con un factor de tono $p \in [-0.5, 0.5]$ (potenciometro, ADC de 12 bits), y la duración del ciclo se reparte entre las notas activas:

$$f_{aj} = f_0 \cdot 2^p, \quad T_{nota} = \frac{T_{ciclo}}{N_{activas}} \quad (8)$$

Cada forma de onda se almacena en una tabla de 256 muestras ($x[m] = 32767 \sin(2\pi m/256)$ en la senoidal). Con $f_s = 44.1$ kHz, el número de muestras por periodo y el índice de lectura son:

$$N_p = \left\lfloor \frac{f_s}{f_{aj}} \right\rfloor, \quad i[n] = \left\lfloor \frac{(n \bmod N_p) \cdot 256}{N_p} \right\rfloor \quad (9)$$

La señal pasa por un filtro de primer orden, con estado persistente entre bloques de 64 muestras y coeficiente $a \in [0, 1]$:

$$y[n] = y[n-1] + a(x[n] - y[n-1]) \quad (10)$$

INTERFAZ DE HARDWARE

Subsistema	Implementación	GPIO
Audio I ² S	BCLK, LRCLK y DATA hacia el PCM5102	4, 5, 18
Notas (C4–C5)	13 pulsadores con alternancia y LED por nota	8, 12–17, 19–21, 35–37
Controles	Velocidad y tono (ADC); modo y onda (botones)	3, 10, 40, 39
LEDs	Cadena de 13 WS2812B	38

Limitaciones documentadas: reproducción bloqueante y `delay()` como antirrebote. Mejoras previstas: máquina de estados con `millis()`, audio por DMA o tarea FreeRTOS, síntesis con acumulador de fase y envolvente ADSR.

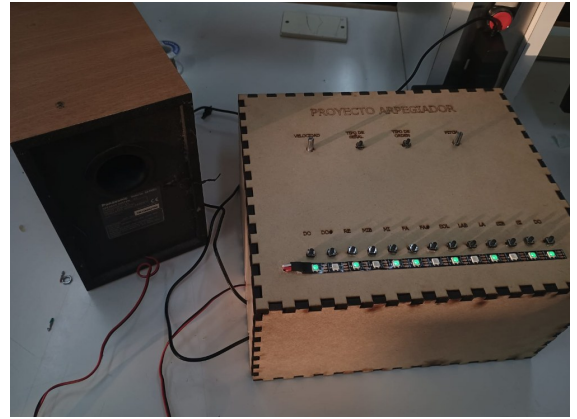


Figura 9: Prototipo final: gabinete de MDF de 5.5 mm (aprox. 25×30×15 cm).

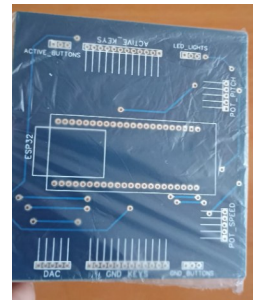


Figura 10: PCB de doble capa.

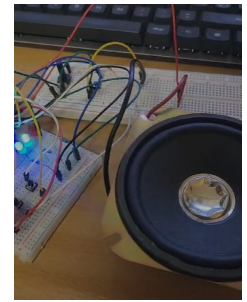


Figura 11: Prueba de DAC y amplificador.

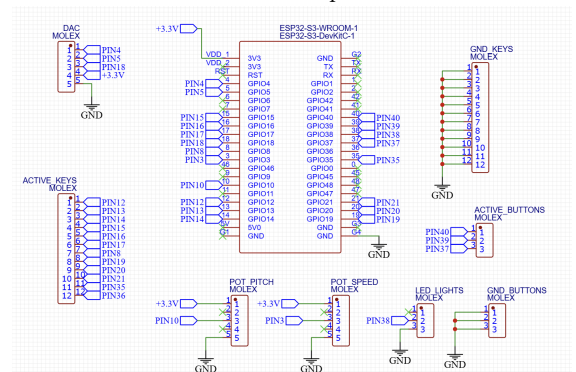


Figura 12: Esquemático con el ESP32-S3 y los conectores.

CONTRIBUCIÓN PERSONAL. Integrante del equipo de cuatro estudiantes (M. Bruschi, R. Yupanqui, E. Torres, J. López). El desarrollo recorrió más de quince iteraciones: prueba del DAC con una senoidal de 400 Hz, prototipo con PWM, migración a I²S, corrección del ruido intermitente mediante el rediseño de la generación de muestras, PCB a medida y gabinete. El repositorio incluye revisión de código y documentación del firmware.